

Projet L2: Simulateur de L-Systèmes

Samuel Banwell, Adjì Lo, Thura Htet, Peace Koutchade,
Nicole Mvembe

Unicaen

26 avril 2026

Section 1

Introduction

Plan

- 1 Introduction
- 2 Comment ça marche ?
 - Représentation de données
 - Interprétation
 - Moteur de règles
 - Affichage
 - Gestion E/S Fichier
- 3 Conclusion

Un L-Système, c'est quoi ?

- Système de réécriture se basant sur un point de départ et des règles de remplacement.
- Utile pour modéliser des structures fractales et biologiques.

Exemple de remplacement

Le système AA , avec la règle $A \rightarrow A + B$, deviendra $A + BA + B$.

Un autre exemple

Courbe de Koch

Axiome : F

Symboles : $\{F\}$

Constantes : $\{+, -\}$

Règles de réécriture : $\{F \rightarrow F + F - F - F + F\}$

Résultat

Après 2 générations...

- ① F
- ② $F + F - F - F + F$
- ③ $F + F - F - F + F + F + F - F - F + F - F + F - F -$
 $F + F - F + F - F - F + F + F + F - F - F + F$

Le système comporte un nombre exponentiel de symboles.

Démonstration

DEMO !

Section 2

Comment ça marche ?

Structure globale

Génération sur plusieurs étapes :

- Gestion des symboles et de l'axiome.
- Interprétation par une tortue graphique (2D/3D).
- Affichage avec un moteur graphique (2D/3D).

L'axiome - le point de départ

L'axiome est le **mot initial** du L-Système, avant toute réécriture.

- Stocké dans une `SymbolList` : liste ordonnée de `Symbol`
- Chaque caractère est converti en objet via la `SymbolFactory`
- Sans axiome, le L-Système ne peut pas démarrer

Symbol — la brique de base

Encapsule un seul char du L-Système

- Implémente `Interpretable3D` → sait s'exécuter sur une tortue
- Implémente `Mot` → peut être vu comme un mot d'une lettre
- `equals()` et `hashCode()` basés sur le char → requis pour les `HashMap`
- Par défaut, `interpret(turtle)` appelle `forward()` → avance et trace un segment

La classe `Constante` hérite de `Symbol` et **redéfinit** `interpret()` pour tourner ou empiler au lieu d'avancer.

Des symboles poids plume

La taille d'une génération croît exponentiellement.

- La courbe de Koch comporte à sa 12^{ème} génération plus de 10 milliards de symboles !
- **Problème** : créer un objet par caractère à chaque fois → catastrophe mémoire
- **Solution** : ne créer qu'un seul objet par char, partagé partout
- Si $c \in "-+*/[]"$ → crée une Constante(c)
- Sinon → crée un Symbol(c)
- Stocké dans un HashMap → **toujours le même objet retourné**

Des symboles poids plume

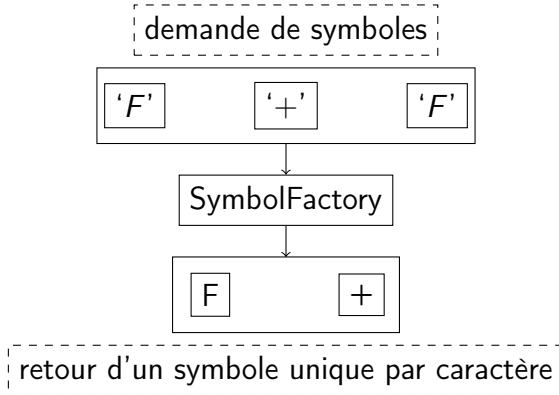


Figure – Principe de fonctionnement de SymbolFactory

Les règles de remplacement



Figure – Remplacement de l'axiome F sur deux générations, avec la règle $F \rightarrow F + F - F - F + F$

Les règles de remplacement



Figure – Remplacement de l'axiome F sur deux générations, avec la règle $F \rightarrow F + F - F - F + F$

Les règles de remplacement

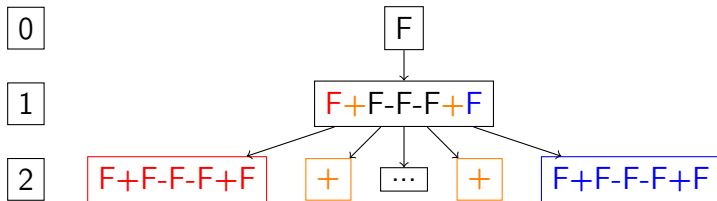


Figure – Remplacement de l'axiome F sur deux générations, avec la règle $F \rightarrow F + F - F - F + F$

Comment le système applique les règles ?

- Toutes les règles sont stockées dans un **RuleSet**
- Pour chaque symbole, on teste toutes les règles
- Chaque règle vérifie si elle est applicable avec `isApplicable()`
- Les règles valides sont sélectionnées
- S'il y en a plusieurs → choix aléatoire ou pondéré
- S'il n'y en a aucune → le symbole reste inchangé

Idée

Une règle transforme toujours : **prédécesseur** → **successeur**

Types de règles (extension du système)

- Règle simple : $A \rightarrow AB$
- Règle stochastique : choix avec probabilité
- Règle contextuelle : dépend des voisins

Dans le code

Le système détecte automatiquement le type de règle via
`RuleSetFactory`

Règle simple : fonctionnement dans le code

- Si le symbole correspond exactement au prédécesseur
- Alors il est remplacé par le successeur

Exemple

$A \rightarrow AB$

- Aucun contexte
- Aucun hasard
- Même résultat à chaque exécution

Dans le code

```
return predecessor.equals(symbol);
```

Un exemple d'interprétation — Tortue 2D

Classe qui lit une `SymbolList` et produit une liste de segments 2D

- Maintient : position (x, y) , angle de direction, une pile pour `[ ]`
- Un nuancier associe chaque lettre à une couleur de tracé
- $F \rightarrow x_2 = x + pas \cdot \cos(\theta), y_2 = y + pas \cdot \sin(\theta) \rightarrow$ segment tracé
- $+ / - \rightarrow$ modifie l'angle θ
- `[ empile  $\{x, y, \theta\}$  - ]` dépile et restaure $\rightarrow$ permet les branches d'arbres

AbstractTurtle3D — le contrat de toute tortue 3D

Classe abstraite : impossible à instancier directement

- Définit 9 méthodes abstract → toute sous-classe **doit** les implémenter
- Stocke la ConfigTortue (pas, angle) → impossible dans une interface Java
- forward() / backward() : avancer ou reculer d'un pas
- rotateU : Yaw - rotation horizontale
- rotateL : Pitch - inclinaison haut/bas
- rotateH : Roll - roulis
- savePosition() / restorePosition() : pile pour []
- interpret(Interpretable3D) : délègue l'action au symbole

Turtle3D — repère mobile et matrices de rotation

En 3D, un seul angle ne suffit pas → la tortue porte un **repère orthonormé à 3 vecteurs**

- `heading[]` : direction de marche, valeur initiale
- `left[]` : coté gauche, valeur initiale
- `up[]` : dessus de la tortue, valeur initiale
- `forward()` : $pos += distance \times heading$ → crée un `Segment3D`
- `rotateU` → tourne `heading` & `left` = Yaw
- `rotateL` → tourne `heading` & `up` = Pitch
- `rotateH` → tourne `left` & `up` = Roll
- `normalize()` après chaque rotation corrige les erreurs d'arrondi

Turtle3D — pile de positions

TurtleState : classe interne qui sauvegarde tout l'état de la tortue

- Stocke pos, heading, left, up avec `.clone()`
- `.clone()` indispensable : sinon la pile pointerait vers les tableaux modifiés
- `[ → stack.push(new TurtleState(pos, heading, left, up))`
- `] → dépile et restaure les 4 tableaux avec .clone()`
- Turtle3D produit une `List<Segment3D>` que la Vue3D convertit en cylindres JavaFX

Un exemple d'interprétation

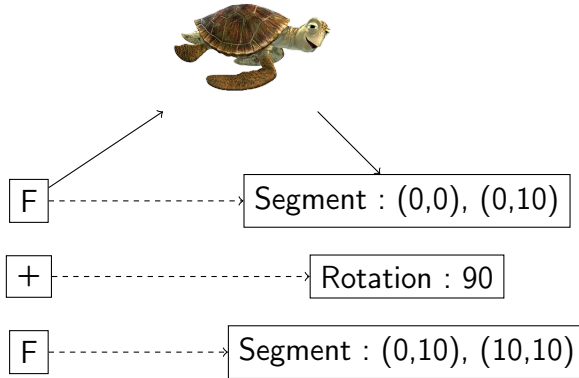


Figure – Illustration avec $F + F$

Vues de l'application

Les vues permettent l'affichage graphique du L-System en 2D et 3D

- **VueLsystem** : affichage 2D des segments générés
- **Vue3D** : visualisation 3D interactive

VueLsystem : affichage 2D

Classe responsable du rendu graphique en 2D.

- Dessine les segments avec `paintComponents()`
- Gestion du zoom avant/arrière
- Changement des couleurs
- Redessin automatique après mis à jour
- Adaptation de la zone de dessin avec scroll

Vue3D : rendu 3D

Représente le L-system dans un espace 3D

- Utilise JavaFX avec une classe qui hérite de Scene
- Reçoit une liste de Segment3D
- Transforme chaque segment en cylindre
- Gestion des couleurs

InterfaceLsystem : coordination des vues

InterfaceLsystem **centralise les interactions utilisateur**

- Création des champs, boutons et panneaux
- Gestion des actions : générer, zoom, couleur, play/pause
- Mise à jour de la vue 2D avec `update2D()`
- Mise à jour de la vue 3D avec `update3D()`
- Intégration de JavaFX dans Swing avec `JFXPanel`

Gestion E/S Fichier

- **Objectifs :**
 - Définir des modèles prédéfinis
 - Enregistrer les modèles utilisateurs
 - Charger les modèles depuis un fichier externe
- **Intérêt :**
 - Étudier et réutiliser des modèles existants
 - Sauvegarder et restaurer le travail utilisateur
 - Modifier les données sans recompilation
- **Principe :**
 - Conversion JSON $\leftrightarrow$ objets Java

Représentation des données

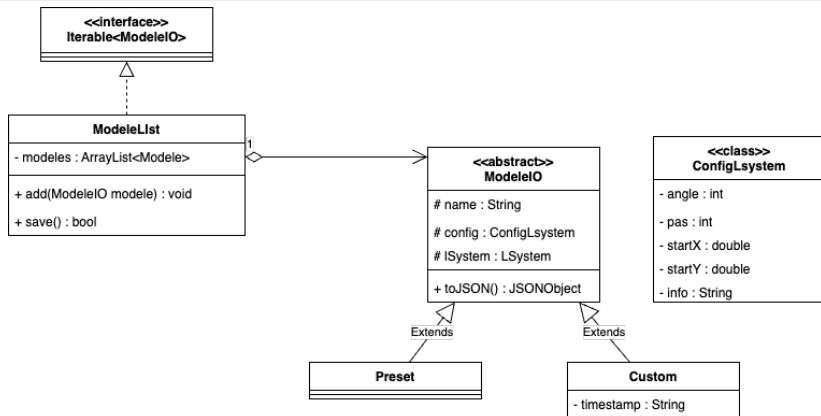


Figure – Diagramme de classe d'un ModeleIO

Structure JSON

Exemple de structure JSON

```
{
  "date": "2026-04-24T15:09:56.661002",
  "name": "Courbe de Koch",
  "axiom": "F",
  "rules": {
    "F": "F+F-F-F+F"
  },
  "config": {
    "pas": 10,
    "angle": 90,
    "startX": 0,
    "startY": 0
  }
}
```

- axiom : état initial du système
- rules : règles de réécriture (clé → symbole)
- config : paramètres d'affichage (géométrie)

Processus de transformation

Fichier JSON $\longrightarrow$ Lecture $\longrightarrow$ Parsing $\longrightarrow$ Objets JAVA

- Lecture du fichier (E/S)
- Conversion JSON $\longrightarrow$ Objets JAVA
- Construction d'un objet `Modele` (Preset ou Custom) en fonction de type de la destination de données

Principe d'instanciation

```
public ModeleList(String pathString) {  
    ...  
    try  
    {  
        for (int i=0; i<modelesArray.length(); i++) {  
            JSONObject modeleObject = modelesArray.getJSONObject(i);  
  
            if (pathString.equals("src/resources/lindenmeyer/ui/presets.json"))  
                this.modeles.add(new Preset(modeleObject));  
            else if (pathString.equals(savePathString))  
                this.modeles.add(new Custom(modeleObject));  
        }  
    }  
    ...  
}
```

- Décision basée sur les données
- Création de l'objet approprié

Utilisation du polymorphisme

- Preset et Custom héritent de Modele
- Manipulation uniforme via une liste :

`List<Modele>`

- Le type concret est transparent pour le reste du programme
- Même interface pour tous les modèles

Limites du système

- Dépendance au format JSON
- Absence de validation des données
- Erreurs détectées uniquement à l'exécution
- Destinations des fichiers `.json` en code dur

Amélioration possible : utilisation d'un mécanisme de Factory comme `SymbolFactory` et `RuleFactory`.

Bilan du projet

- Moteur complet et extensible.
- Rendu 2D et 3D fonctionnel.
- Interface utilisateur interactive et robuste.